System and Method for Conversational Booking via Real-Time API-Driven Search Expansion

Category: Software

BACKGROUND

The present disclosure relates generally to computer-implemented booking systems, and more specifically to autonomous search expansion and response synthesis in conversational vacation rental platforms.Currently, vacation rental booking typically involves users manually navigating through multiple steps to find available properties. Users initially specify their desired dates and location, and the system queries property management systems (PMS) for availability. If the initial search yields no results or only partially fulfills the request, users are forced to manually adjust their search parameters, such as modifying the date range or location, and resubmit the query. This iterative process of manual calendar navigation and search refinement is computationally expensive, requiring multiple API calls and data processing cycles for each user interaction. Traditional systems lack the ability to intelligently adapt the search based on real-time availability data. Furthermore, existing systems often present availability information in a rigid, calendar-based format, requiring users to visually scan and interpret complex data. This manual interpretation introduces latency and cognitive overhead, leading to a disjointed and frustrating user experience.Conventional template-based automation platforms offer some degree of workflow automation, but are fundamentally limited by their reliance on predefined templates and rigid pre-configuration. These platforms require users to explicitly define the data mappings and logic for each step in the workflow. This approach is computationally expensive, as it necessitates manual design and maintenance of complex workflow definitions. Furthermore, these platforms lack the ability to autonomously adapt to changing conditions or unexpected events. They are fragile to schema changes in underlying APIs and require significant manual intervention to update and maintain workflows. The lack of semantic context in these platforms also limits their ability to intelligently interpret user intent and adapt the workflow accordingly.Traditional Robotic Process Automation (RPA) tools automate repetitive tasks by mimicking human interactions with software applications. While RPA can automate certain aspects of the booking process, such as data entry and form filling, it is computationally expensive and fragile. RPA relies on precise screen coordinates and UI element recognition, making it susceptible to even minor changes in the application interface. Furthermore, RPA lacks the ability to reason about the data it is processing or to adapt to changing conditions. It requires rigid pre-configuration and is unable to autonomously synthesize workflows based on user behavior or real-time data. The high latency associated with RPA, due to its reliance on screen scraping and UI automation, further limits its applicability in real-time conversational booking scenarios.AI assistants and copilots offer

natural language interfaces for interacting with software applications. However, existing AI assistants typically rely on predefined commands and limited natural language understanding. They lack the ability to autonomously synthesize complex workflows or to adapt to changing conditions. Furthermore, existing AI assistants often require explicit user programming to define the logic for each task. They are unable to infer cross-application workflows based on user behavior or to personalize the experience without explicit configuration. The lack of semantic action abstraction in these assistants also limits their ability to intelligently interpret user intent and to execute complex tasks.Low-code/no-code platforms aim to simplify the development of software applications by providing visual interfaces and pre-built components. However, these platforms are often limited in their expressiveness and flexibility. They may not support the full range of functionality required for complex booking scenarios. Furthermore, low-code/no-code platforms often generate code that is less efficient and less maintainable than hand-written code. They also lack the ability to autonomously synthesize workflows or to adapt to changing conditions.Script-based automation provides a more flexible approach to automating tasks, but requires users to have programming skills. Scripting languages such as Python or JavaScript can be used to automate various aspects of the booking process. However, script-based automation requires significant manual effort to design, implement, and maintain. Furthermore, script-based automation lacks the ability to autonomously synthesize workflows or to adapt to changing conditions.Enterprise integration platforms (EIPs) facilitate the integration of disparate systems and applications within an organization. While EIPs can be used to connect various components of the booking process, they are typically complex and expensive to implement. Furthermore, EIPs often require significant manual configuration and customization. They lack the ability to autonomously synthesize workflows or to adapt to changing conditions.Therefore, there is a need for a system that can intelligently and autonomously expand search parameters based on real-time availability data and synthesize a comprehensive response within a single conversational turn, without requiring manual user intervention or iterative search refinements.

SUMMARY OF THE INVENTION

To address the limitations of the prior art, the present disclosure provides a system and method for conversational booking via real-time API-driven search expansion. In one aspect, the method comprises receiving a natural language booking request, executing a real-time query against a Property Management System (PMS) API based on the request, analyzing an availability response array received from the PMS API, classifying unavailability as either partial or complete based on the analysis, and autonomously triggering a conditional search expansion strategy based on the classification, without requiring user re-specification of search parameters. This triggering involves re-querying the PMS API with a narrow expansion window for partial unavailability or a broad expansion window for complete unavailability. Finally, the method synthesizes a natural language response incorporating results from the re-querying, all

performed within a single conversational turn. By autonomously adapting the search based on real-time availability data and synthesizing a comprehensive response within a single conversational turn, the system eliminates the need for manual user intervention or iterative search refinements.The core innovation lies in a progressive conversational search algorithm that uniquely integrates real-time Property Management System (PMS) API query execution with natural language interactions. This algorithm analyzes the availability response array from PMS API queries and classifies unavailability as either "partial" or "complete." Based on this classification, it dynamically and conditionally triggers distinct search expansion strategies, such as a narrow expansion window for partial unavailability and a broad expansion window for complete unavailability. This autonomous, single-conversational-turn classification-to-expansion logic eliminates the need for user re-specification of search parameters, providing a seamless and efficient user experience within a single conversational turn. The system also features a robust mechanism for normalizing disparate and heterogeneous PMS API response data into a standardized internal schema and transforms property documentation uploaded by managers into vector embeddings. The system performs cosine similarity searches against these embeddings, enabling highly relevant property matching based on semantic understanding rather than keyword matching. Geographic coordinates configured by property managers are utilized for classification, enabling the dynamic integration and retrieval of location-based contextual services relevant to the property and user's query. All critical operations including search expansion, context retrieval, and response synthesis are executed entirely within a single request-response cycle.This system offers several key advantages over existing solutions. It eliminates the manual workflow design burden by autonomously discovering optimal search strategies based on real-time API responses. It provides semantic abstraction by understanding user intent from natural language interactions and adapting the search accordingly. The system achieves cross-application synthesis by chaining actions across disparate PMS APIs and location-based services. It offers personalization without configuration by adapting to individual user preferences based on their search history and behavior. The system enables de novo workflow creation by autonomously synthesizing new search strategies based on real-time data and user interactions, rather than relying on predefined templates. By eliminating manual calendar navigation and iterative search refinements, the system significantly enhances vacation rental booking efficiency and provides a seamless and efficient user experience.The following detailed description provides comprehensive technical specifications and implementation details of the system and method for conversational booking via real-time API-driven search expansion.

DETAILED DESCRIPTION

The System Environment (100) illustrates a networked computing architecture configured to execute conversational booking operations via real-time API-driven search expansion. The

System Environment (100) generally comprises at least one User Device (102) communicatively coupled to a Server System (106) via a Network (104). The User Device (102) is a physical computing device, such as a smartphone, tablet, laptop, or desktop computer, comprising a user-interface display, a processor, and a network transceiver. The User Device (102) is configured to execute a client application or web browser that captures unstructured natural language text input from a user and renders natural language responses received from the Server System (106). The Network (104) represents a data communication pathway, such as the Internet, a cellular network (e.g., 4G, 5G), or a Wide Area Network (WAN), utilizing standard transport protocols including TCP/IP, HTTP/S, and WebSocket protocols to facilitate secure data transmission.The Server System (106) acts as the central processing hub of the invention and is defined as a high-performance computing apparatus or a distributed cloud computing cluster. The Server System (106) comprises physical hardware components including at least one Processor (108), a Network Interface (112), and a Non-Transitory Computer-Readable Memory (110). The Processor (108) may be a Central Processing Unit (CPU), a Graphics Processing Unit (GPU) optimized for vector calculations, or a Tensor Processing Unit (TPU). The Network Interface (112) manages inbound and outbound data packets between the Server System (106) and the Network (104). The Memory (110) stores the operating system, database management systems, and the specific computer-executable instructions that constitute the functional modules of the present disclosure.Stored within the Memory (110) and executed by the Processor (108) is an API Gateway (114). The API Gateway (114) serves as the entry point for the system, configured to receive incoming HTTP POST requests containing user queries from the User Device (102), perform load balancing, and route requests to an Orchestration Layer (116). The Orchestration Layer (116) is a software logic controller that manages the state of the single request-response cycle. It is configured to instantiate and coordinate the execution of subsequent modules in a specific sequence based on conditional logic branches. The Orchestration Layer (116) maintains a transient session cache, utilizing technologies such as Redis, to hold the query context, intermediate API responses, and classification states during the processing of a single conversational turn.The Memory (110) further comprises a Natural Language Processing (NLP) Module (118). The NLP Module (118) includes parsing algorithms configured to extract temporal entities from the unstructured text received from the User Device (102). It normalizes relative date phrases (e.g., "next weekend") into specific ISO 8601 date ranges. The NLP Module (118) passes these normalized date parameters to an API Integration Module (120). The API Integration Module (120) is communicatively coupled to a Credential Vault (136), which is a secure storage partition containing encrypted authentication keys and tokens for a plurality of External PMS Platforms (140). The API Integration Module (120) is configured to select the appropriate credentials and execute real-time, synchronous queries against the Application Programming Interfaces (APIs) of the External PMS Platforms (140) to retrieve availability data.To handle the heterogeneity of data returned by different External PMS Platforms (140), the Server System

(106) executes a Data Normalization Module (122). This module comprises a library of JavaScript transformation functions mapped to specific PMS providers. The Data Normalization Module (122) receives raw JSON or XML responses from the API Integration Module (120) and transforms them into a Standardized Internal Schema. This schema is a rigid JSON object structure containing specific fields: date, status (enumerated as AVAILABLE, BOOKED, or BLOCKED), price, currency, and minimum stay requirements. This normalization ensures that subsequent algorithmic processing is agnostic to the source of the data.The core logic of the invention is encapsulated in the Progressive Search Algorithm Module (124), stored in the Memory (110). This module contains the conditional logic instructions for analyzing the normalized availability array generated by the Data Normalization Module (122). The Progressive Search Algorithm Module (124) is configured to count date objects with an "AVAILABLE" status. It implements a decision tree where: (1) if the count equals the requested duration, the workflow proceeds; (2) if the count is greater than zero but less than the requested duration, it flags "Partial Unavailability" and triggers a narrow expansion subroutine (e.g., ±7 days); and (3) if the count is zero, it flags "Complete Unavailability" and triggers a broad expansion subroutine (e.g., ±30 days). These subroutines instruct the Orchestration Layer (116) to re-invoke the API Integration Module (120) with adjusted parameters within the same processing cycle.Parallel to the availability check, the Server System (106) executes a Vector Embedding Module (126) and a Semantic Search Module (128). The Vector Embedding Module (126) is configured to ingest property documentation (e.g., PDF, TXT files), chunk the text into tokenized segments, and generate 1536-dimensional numerical vector arrays using a transformer model. These vectors are stored in a Vector Database (134), such as a PostgreSQL database extended with pgvector. The Semantic Search Module (128) executes cosine similarity searches against the Vector Database (134) using the vector representation of the user's query to retrieve semantically relevant text chunks (e.g., specific house rules or amenities) that match the user's intent, even if exact keywords are absent.Furthermore, the Memory (110) stores a Geographic Classification Module (130). This module holds the geographic coordinates (latitude and longitude) of the properties and is communicatively coupled via the Network Interface (112) to External Location Services (142). The Geographic Classification Module (130) executes rule-based distance calculations against data retrieved from the External Location Services (142), such as coastline boundaries, elevation maps, and population density databases. Based on these calculations, the module assigns a location context (e.g., "Coastal," "Mountain," "Urban") to the property, which determines which subset of local attraction data is relevant to the user's request.Finally, the System (100) includes a Response Synthesis Module (132). This module is an interface to a Large Language Model (LLM). It is configured to aggregate the outputs from the Progressive Search Algorithm Module (124) (including the results of any autonomous search expansions), the Semantic Search Module (128) (retrieved context chunks), and the Geographic Classification Module (130). The Response Synthesis Module (132) constructs a prompt containing these aggregated data

points and instructs the LLM to generate a coherent, natural language response. This synthesized response is transmitted back through the API Gateway (114) to the User Device (102), completing the single request-response cycle.The System Environment (100) relies on a strict hierarchy of data structures and schemas stored within the Non-Transitory Computer-Readable Memory (110) and the associated databases to facilitate the real-time transformation of unstructured natural language into executable booking actions. The primary input data structure is the "Conversational Request Object," which is instantiated by the API Gateway (114) upon receipt of an HTTP POST payload from the User Device (102). This object is serialized in JavaScript Object Notation (JSON) format and comprises a "Session_ID" (a 128-bit Universally Unique Identifier), a "Timestamp" (formatted in ISO 8601), a "User_ID," and a "Raw_Query_Payload" field containing the unstructured text string input by the user. As the data matures through the Natural Language Processing (NLP) Module (118), this object is appended with a "Temporal_Entity_Map," a nested JSON object containing normalized "Start_Date" and "End_Date" fields, enabling the system to convert relative phrases such as "next weekend" into concrete query parameters without altering the original raw input.To manage the heterogeneity of external data sources, the Data Normalization Module (122) generates a "Standardized Availability Schema," which serves as the intermediate data state between the external PMS Platforms (140) and the internal Progressive Search Algorithm Module (124). This schema is defined as a rigid array of "Daily Availability Objects." Each Daily Availability Object contains specific typed fields: "Date_Index" (Date format), "Status_Enum" (restricted to values: AVAILABLE, BOOKED, BLOCKED), "Price_Value" (Decimal/Float), "Currency_Code" (ISO 4217 string), and "Min_Stay_Constraint" (Integer). This standardization ensures that the logic within the Server System (106) remains agnostic to the specific XML or JSON formatting variations inherent to the different External PMS Platforms (140). These objects are temporarily stored in the transient session cache of the Orchestration Layer (116) during the request-response cycle.The Vector Database (134) stores "Semantic Embedding Objects" utilized by the Semantic Search Module (128). These objects represent the transformed state of the property documentation. Each Semantic Embedding Object comprises a "Vector_ID" (Primary Key), a "Property_Reference_ID" (Foreign Key linking to the property), a "Text_Chunk_Payload" (VarChar containing the raw text segment, approx. 500 tokens), and a "Vector_Array." The Vector_Array is a 1536-dimensional array of 32-bit floating-point numbers representing the semantic meaning of the text chunk, generated by the transformer model within the Vector Embedding Module (126). These objects are indexed using Hierarchical Navigable Small World (HNSW) graphs to facilitate rapid Approximate Nearest Neighbor (ANN) search operations, allowing the system to retrieve context based on cosine similarity rather than keyword matching.The logic governing the autonomous search expansion is encapsulated in a "Search Execution State DAG (Directed Acyclic Graph)," managed by the Orchestration Layer (116). This data structure tracks the maturity of the search operation. It includes a

"Current_Iteration_Count" (Integer), a "Search_Window_Parameters" object (defining the current date range being queried), and a "State_Flag" (Enumerated: INITIAL_QUERY, NARROW_EXPANSION, BROAD_EXPANSION, COMPLETE). The Progressive Search Algorithm Module (124) reads this state object to determine the next workflow step. For instance, if the "Availability_Count" derived from the Standardized Availability Schema is zero, the algorithm updates the "State_Flag" to BROAD_EXPANSION and modifies the "Search_Window_Parameters" to ±30 days, triggering a recursive execution loop within the Orchestration Layer (116).To secure authentication against the External PMS Platforms (140), the Credential Vault (136) stores "Encrypted Credential Objects." These objects utilize Advanced Encryption Standard (AES-256) encryption at rest. Each object contains a "Platform_Identifier," an "API_Key," a "Client_Secret," and an "OAuth_Token" with an associated "Expiration_Timestamp." The API Integration Module (120) retrieves and decrypts these objects into memory only at the moment of request execution. Furthermore, the Geographic Classification Module (130) utilizes a "Geo-Context Object," which maps "Property_Coordinates" (Latitude/Longitude in Decimal Degrees) to a "Context_Tag_Array" (e.g., ["Coastal", "Urban"]). This object is generated by processing the coordinates against spatial databases stored in the Memory (110) or retrieved from External Location Services (142), defining the environmental context used by the Response Synthesis Module (132) to construct the final natural language response.The operational workflow commences when the User Device (102) transmits a Hypertext Transfer Protocol Secure (HTTPS) POST request across the Network (104) to the Server System (106). This request contains a JSON payload comprising the user s unstructured natural language query. The API Gateway (114) intercepts this incoming transmission, performs initial transport layer security (TLS) termination, and instantiates a Conversational Request Object. This object is immediately routed to the Orchestration Layer (116), which initializes a transient session cache in the Memory (110) to maintain the state of the request-response cycle. Upon initialization, the Orchestration Layer (116) invokes the Natural Language Processing (NLP) Module (118). The NLP Module (118) executes Named Entity Recognition (NER) algorithms to parse the Raw_Query_Payload field, extracting temporal entities and normalizing relative phrases such as "next weekend" or "Christmas week" into specific ISO 8601 Start_Date and End_Date values, which are appended to the Conversational Request Object.Subsequent to temporal extraction, the Orchestration Layer (116) triggers the API Integration Module (120) to retrieve availability data. The API Integration Module (120) accesses the Credential Vault (136) to retrieve and decrypt the Encrypted Credential Objects associated with the target External PMS Platforms (140). Using the decrypted API keys and OAuth tokens, the API Integration Module (120) constructs and executes synchronous HTTP GET requests against the heterogeneous endpoints of the External PMS Platforms (140). Upon receiving the raw XML or JSON responses, the system passes the data to the Data Normalization Module (122). This module applies provider-specific JavaScript transformation functions to map the disparate external data

fields into the Standardized Availability Schema. This process generates a rigid array of Daily Availability Objects, ensuring that the status of every requested date is uniformly categorized as AVAILABLE, BOOKED, or BLOCKED within the internal system logic.Once the data is normalized, the Orchestration Layer (116) invokes the Progressive Search Algorithm Module (124) to execute the core availability logic. The module analyzes the Standardized Availability Schema by iterating through the array of Daily Availability Objects and calculating an Availability_Count. The algorithm compares this count against the requested stay duration derived from the NLP Module (118). If the Availability_Count equals the requested duration, the system flags the request as successful and proceeds to response synthesis. However, if the Availability_Count is greater than zero but less than the requested duration, the module identifies a "Partial Unavailability" condition. Responsive to this determination, the Progressive Search Algorithm Module (124) updates the Search Execution State DAG to a NARROW_EXPANSION state. This triggers a recursive instruction to the Orchestration Layer (116) to re-invoke the API Integration Module (120) with a modified date range expanded by exactly seven days (±7 days) from the original request.Alternatively, if the Progressive Search Algorithm Module (124) determines that the Availability_Count is zero, it identifies a "Complete Unavailability" condition. The module then updates the Search Execution State DAG to a BROAD_EXPANSION state. This state change instructs the Orchestration Layer (116) to execute a broader search subroutine, re-querying the External PMS Platforms (140) with a date range expanded by thirty days (±30 days). These search expansions occur autonomously within the single processing cycle, without requiring additional input from the User Device (102). The Data Normalization Module (122) processes the results of any expansion queries, appending the new availability data to the session cache.Simultaneously with the availability processing, the Orchestration Layer (116) executes parallel context retrieval operations. The Semantic Search Module (128) generates a vector representation of the user's query using the transformer model and executes a cosine similarity search against the Vector Database (134). This operation traverses the Hierarchical Navigable Small World (HNSW) graphs to identify and retrieve Semantic Embedding Objects such as specific house rules or amenity descriptions that possess a high semantic correlation to the user's intent. Concurrently, the Geographic Classification Module (130) retrieves the property s fixed coordinates and queries the External Location Services (142). By calculating distances to predefined geographical features, the module generates a Geo-Context Object (e.g., assigning a "Coastal" tag if within 5km of a coastline), which is stored in the session cache to inform the tone and content of the final response.In the final phase of the workflow, the Orchestration Layer (116) passes the aggregated data comprising the final Standardized Availability Schema (including any expansion results), the retrieved Semantic Embedding Objects, and the Geo-Context Object to the Response Synthesis Module (132). This module constructs a complex prompt context and interfaces with the Large Language Model (LLM). The LLM generates a natural language response that articulates the availability status, suggests alternative dates if an expansion

occurred, and weaves in the semantic and geographic context relevant to the user's query. The Response Synthesis Module (132) transmits this synthesized text back to the API Gateway (114), which packages it into an HTTP response and transmits it to the User Device (102), thereby concluding the single request-response cycle.One of ordinary skill in the art will appreciate that the specific system architecture described above is merely exemplary, and numerous alternative embodiments exist that fall within the scope of this disclosure. While described as a cloud-based server system, the Server System (106) may be implemented on edge devices, a local intranet, a peer-to-peer network, or a blockchain distributed ledger. The processing need not occur on a single device; steps may be distributed such that certain steps occur on the User Device (102) while others occur on a remote server or a cluster of servers.In alternative embodiments, the Server System (106) can be deployed in a pure cloud infrastructure, an on-premise deployment within an enterprise data center, a hybrid architecture with monitoring agents on user devices and processing in the cloud, an edge computing environment where processing occurs closer to the users, or a peer-to-peer distributed network with no central server. In a pure cloud infrastructure, all components reside on remote servers, offering scalability and accessibility. An on-premise deployment involves hosting the system within an enterprise data center, providing greater control over data and security. A hybrid architecture leverages both local user devices and cloud resources, optimizing performance and resource utilization. Edge computing places processing closer to users, reducing latency and bandwidth consumption. A peer-to-peer distributed network eliminates the need for a central server, enhancing resilience and decentralization.The User Device (102) is not limited to the specific examples provided. It can be a desktop system (running Windows, macOS, or Linux), a mobile device (running iOS or Android), an embedded system, or a browser-based application. Mobile devices may utilize platform-specific monitoring approaches. The system may operate in mixed heterogeneous environments, where different components run on different types of devices.Hardware acceleration can be employed to enhance performance. GPU acceleration can be used for machine learning inference, while TPUs or specialized AI accelerators can further optimize AI-related tasks. FPGAs can provide low-latency pattern matching capabilities. Distributed computing clusters can be used to handle large-scale processing requirements.While the primary embodiment might use Python for certain modules, the implementation language is not limiting. Compiled languages such as Go, Rust, or C++ can be used for performance-critical components. JVM languages like Java, Kotlin, or Scala can be used for enterprise integration. JavaScript or TypeScript can be used for web-based components. A microservices architecture can be adopted, with different services implemented in different languages. One of ordinary skill in the art will appreciate that the choice of programming language is a design choice based on factors such as performance requirements, existing infrastructure, and developer expertise.The machine learning frameworks used in the Vector Embedding Module (126) and Semantic Search Module (128) are also not limited to the specific examples provided. TensorFlow can be used for large-scale neural networks, PyTorch

for research and development, JAX for high-performance computing, Scikit-learn for traditional ML algorithms, or custom implementations of algorithms. The choice of framework depends on the specific requirements of the application and the available resources.Reference to a relational database is exemplary; key-value stores, graph databases, time-series databases, or vector indexes may be utilized. Relational databases (e.g., PostgreSQL, MySQL) are suitable for structured data, while NoSQL databases (e.g., MongoDB, Cassandra) offer flexible schemas. Time-series databases (e.g., InfluxDB, TimescaleDB) are optimized for event streams. Graph databases (e.g., Neo4j) are useful for relationship tracking. Vector databases (e.g., Pinecone, Weaviate) are designed for storing and querying embeddings.The pattern recognition techniques used in the NLP Module (118) and the Progressive Search Algorithm Module (124) are not limited to the specific examples provided. Statistical methods such as Markov chains, Hidden Markov Models, and Bayesian networks can be used. Deep learning techniques such as transformer architectures, LSTM networks, and GRU networks can be used. Traditional ML techniques such as random forests, gradient boosting, and decision trees can be used. Sequence mining algorithms such as PrefixSpan, SPADE, and CloSpan can be used. Hybrid approaches combining multiple techniques can also be used.Workflow optimization can be achieved through various methods. Reinforcement learning can be used for workflow improvement. Genetic algorithms can be used for workflow synthesis. Simulated annealing can be used for parameter optimization. Constraint satisfaction can be used for workflow validation.Applications can be integrated through various means. API-based integration (using REST, GraphQL, or gRPC) can be used. UI automation (using Selenium, Playwright, or Puppeteer) can be used. Native SDKs and libraries can be used. Browser extensions can be used. Hybrid approaches combining multiple methods can also be used.While a Neural Network is described for the Vector Embedding Module (126), the logic may be implemented via heuristic rules, decision trees, support vector machines, or regression analysis. The specific choice of algorithm is dependent on the corpus of property documentation, the computational resources available, and the desired tradeoff between accuracy and speed.The API Integration Module (120) is described as executing synchronous HTTP GET requests. In alternative embodiments, asynchronous requests, message queues (e.g., Kafka, RabbitMQ), or serverless functions (e.g., AWS Lambda, Google Cloud Functions) may be employed to improve scalability and resilience.The Data Normalization Module (122) is described as using JavaScript transformation functions. One of ordinary skill in the art will appreciate that other scripting languages (e.g., Python, Lua) or compiled languages (e.g., Java, Go) could be used to implement the data transformation logic. The key inventive aspect is the mapping of heterogeneous data fields into a Standardized Internal Schema, regardless of the specific implementation language.The Geographic Classification Module (130) is described as using rule-based distance calculations. In alternative embodiments, machine learning models (e.g., classification algorithms trained on geographic data) could be used to assign location contexts to properties.The Response Synthesis Module (132) is described as interfacing with a

Large Language Model (LLM). One of ordinary skill in the art will appreciate that other natural language generation techniques (e.g., template-based generation, rule-based generation) could be used. Furthermore, the specific LLM used is not limiting; alternative LLMs (e.g., GPT-3, LaMDA, open-source models) could be used.Any processor configured to perform the steps described herein is within the scope of this disclosure, regardless of architecture (x86, ARM, RISC-V, FPGA, ASIC). The functions performed by the various modules may be combined or further subdivided without departing from the scope of the invention. The specific data structures and file formats described are merely exemplary; alternative data structures and file formats may be used.

RAMIFICATIONS AND SCOPE

Ramifications and Scope: Use CasesWhile the detailed description focuses on a vacation rental booking system, the core innovation of a progressive conversational search algorithm, particularly its autonomous classification-to-expansion logic and heterogeneous data normalization, extends far beyond this specific application. The system's ability to intelligently adapt search parameters based on real-time API response classification, eliminate iterative user re-querying, and seamlessly integrate diverse data sources makes it applicable across a wide range of industries and functional domains.In alternative embodiments, the system can be implemented using various programming languages and frameworks. While Python may be suitable for prototyping and certain modules, performance-critical components can be implemented in compiled languages such as Go, Rust, or C++. For enterprise integration, JVM languages like Java, Kotlin, or Scala can be employed. Web-based components can leverage JavaScript or TypeScript. The choice of programming language is a design choice based on factors such as performance requirements, existing infrastructure, and developer expertise, and does not alter the fundamental inventive concept. Similarly, the specific machine learning frameworks used in the Vector Embedding Module and Semantic Search Module are not limiting. TensorFlow, PyTorch, JAX, Scikit-learn, or custom implementations of algorithms can be used depending on the specific requirements of the application and available resources.The data storage mechanisms can also be varied. While the primary embodiment might use a relational database, key-value stores, graph databases, time-series databases, or vector indexes may be utilized depending on the data characteristics and performance requirements. Relational databases (e.g., PostgreSQL, MySQL) are suitable for structured data, while NoSQL databases (e.g., MongoDB, Cassandra) offer flexible schemas. Time-series databases (e.g., InfluxDB, TimescaleDB) are optimized for event streams. Graph databases (e.g., Neo4j) are useful for relationship tracking. Vector databases (e.g., Pinecone, Weaviate) are designed for storing and querying embeddings.The system can be deployed in various infrastructure environments. These include a pure cloud infrastructure, an on-premise deployment within an enterprise data center, a hybrid architecture with monitoring agents on user devices and

processing in the cloud, an edge computing environment where processing occurs closer to the users, or a peer-to-peer distributed network with no central server. The hosting model does not change the core invention.The system's architecture allows for adaptation across different scales. In a consumer context, it can be deployed as a single-user mode on a personal device for personal productivity optimization. For team or workgroup automation in small organizations, it can be deployed on a local server. For department-level deployment in medium-scale organizations, it can be deployed on a dedicated server cluster. For enterprise-wide deployment in large-scale organizations, it can be deployed on a distributed cloud infrastructure. Finally, it can be offered as a multi-tenant SaaS architecture serving thousands of concurrent organizations.The system can be deployed across various platforms, including desktop operating systems (Windows, macOS, Linux), mobile platforms (iOS, Android), web browsers, embedded systems and IoT devices, and mixed-platform environments.Beyond vacation rentals, the core innovation can be applied to numerous industry-specific applications.In healthcare, the system can be used to automate patient appointment scheduling, intelligently searching for available slots based on doctor availability, resource allocation, and patient preferences. The progressive search algorithm can handle complex constraints such as doctor specialization, equipment availability, and room capacity, autonomously expanding the search window to find suitable appointments without requiring manual intervention from the user. Furthermore, it can normalize data from heterogeneous Electronic Health Record (EHR) systems, such as HL7 data, to ensure compatibility and seamless integration.In finance, the system can be used to automate compliance checks and fraud detection. The progressive search algorithm can analyze ledger entries and transaction data, identifying patterns and anomalies that may indicate fraudulent activity. The system can autonomously expand the search to investigate related transactions and accounts, providing a comprehensive view of potential fraud. It can also normalize data from different financial systems, such as banking systems, trading platforms, and payment gateways, to ensure consistent analysis.In logistics, the system can be used to optimize supply chain tracking and delivery scheduling. The progressive search algorithm can analyze real-time data from various sources, such as GPS trackers, inventory management systems, and weather forecasts, to identify potential delays and disruptions. The system can autonomously adjust delivery routes and schedules to minimize the impact of these disruptions, ensuring timely delivery of goods. It can also normalize data from different logistics providers, such as shipping companies, trucking companies, and warehouses, to provide a unified view of the supply chain.In cybersecurity, the system can be used to automate threat detection and incident response. The progressive search algorithm can analyze threat detection logs and network traffic data, identifying potential security breaches and vulnerabilities. The system can autonomously investigate suspicious activity, expanding the search to identify related events and compromised systems. It can also normalize data from different security tools, such as firewalls, intrusion detection systems, and antivirus software, to provide a comprehensive security posture.Beyond these industry-specific

applications, the system can be applied to various functional categories.In HR, the system can be used to automate employee onboarding workflows, streamlining the process of collecting employee information, assigning training modules, and granting access to necessary systems.In DevOps, the system can be used to orchestrate CI/CD pipelines, automating the process of building, testing, and deploying software applications.In legal, the system can be used to automate contract review, identifying potential risks and liabilities based on pre-defined rules and regulations.The system can also be applied to cross-industry patterns, such as any multi-step process involving multiple applications, any repetitive task with observable patterns, and any workflow requiring data coordination between systems.The pattern recognition techniques used in the NLP Module and the Progressive Search Algorithm Module are not limited to the specific examples provided. Statistical methods such as Markov chains, Hidden Markov Models, and Bayesian networks can be used. Deep learning techniques such as transformer architectures, LSTM networks, and GRU networks can be used. Traditional ML techniques such as random forests, gradient boosting, and decision trees can be used. Sequence mining algorithms such as PrefixSpan, SPADE, and CloSpan can be used. Hybrid approaches combining multiple techniques can also be used.Workflow optimization can be achieved through various methods. Reinforcement learning can be used for workflow improvement. Genetic algorithms can be used for workflow synthesis. Simulated annealing can be used for parameter optimization. Constraint satisfaction can be used for workflow validation.Applications can be integrated through various means. API-based integration (using REST, GraphQL, or gRPC) can be used. UI automation (using Selenium, Playwright, or Puppeteer) can be used. Native SDKs and libraries can be used. Browser extensions can be used. Hybrid approaches combining multiple methods can also be used.While a Neural Network is described for the Vector Embedding Module, the logic may be implemented via heuristic rules, decision trees, support vector machines, or regression analysis. The specific choice of algorithm is dependent on the corpus of property documentation, the computational resources available, and the desired tradeoff between accuracy and speed.The API Integration Module is described as executing synchronous HTTP GET requests. In alternative embodiments, asynchronous requests, message queues (e.g., Kafka, RabbitMQ), or serverless functions (e.g., AWS Lambda, Google Cloud Functions) may be employed to improve scalability and resilience.The Data Normalization Module is described as using JavaScript transformation functions. One of ordinary skill in the art will appreciate that other scripting languages (e.g., Python, Lua) or compiled languages (e.g., Java, Go) could be used to implement the data transformation logic. The key inventive aspect is the mapping of heterogeneous data fields into a Standardized Internal Schema, regardless of the specific implementation language.The Geographic Classification Module is described as using rule-based distance calculations. In alternative embodiments, machine learning models (e.g., classification algorithms trained on geographic data) could be used to assign location contexts to properties.The Response Synthesis Module is described as interfacing with a Large Language Model (LLM). One of

ordinary skill in the art will appreciate that other natural language generation techniques (e.g., template-based generation, rule-based generation) could be used. Furthermore, the specific LLM used is not limiting; alternative LLMs (e.g., GPT-3, LaMDA, open-source models) could be used.The specific configurations, choice of materials, and the size and shape of various elements can be varied according to particular design specifications or functional requirements. The detailed description is to be construed as exemplary only and not as limiting the invention.

CLAIMS

1. A computer-implemented method for enhancing vacation rental booking efficiency, comprising:receiving, by a processor, a natural language booking request;executing, by said processor, a real-time query against a Property Management System (PMS) API based on said request;analyzing, by said processor, an availability response array received from said PMS API;classifying, by said processor, unavailability as one of partial unavailability or complete unavailability based on said analysis;autonomously triggering, by said processor, a conditional search expansion strategy based on said classification, without requiring user re-specification of search parameters, wherein said triggering comprises:re-querying said PMS API with a narrow expansion window for said partial unavailability; orre-querying said PMS API with a broad expansion window for said complete unavailability;synthesizing, by said processor, a natural language response incorporating results from said re-querying, said analyzing, said classifying, said triggering, and said synthesizing all performed within a single conversational turn.

1.1. The method of claim 1, wherein classifying unavailability comprises: counting date objects within said availability response array where a status field equals "AVAILABLE"; classifying said unavailability as partial unavailability when said count is greater than zero but less than a total number of requested nights; and classifying said unavailability as complete unavailability when said count equals zero.

1.2. The method of claim 1, wherein said narrow expansion window is a date range of ±7 days from an original date range of said request, and wherein said broad expansion window is a date range of ±30 days from said original date range.

1.3. The method of claim 1, further comprising: normalizing said availability response array from said PMS API into a standardized internal schema prior to said analyzing; and wherein said conditional search expansion strategy is implemented via conditional execution paths within an orchestration layer based on an IF-THEN evaluation of said availability classification.

2. A system comprising a processor and a memory, the system configured to:a. receive a natural language booking request from a user;b. execute a real-time query against a Property Management System (PMS) API based on said request;c. normalize an availability response

array from said PMS API into a standardized internal schema;d. classify, by a progressive search algorithm module, unavailability within said normalized availability response array as one of partial unavailability or complete unavailability;e. conditionally trigger, by an orchestration layer, a search expansion strategy based on said classification, wherein said conditional triggering is performed within a single conversational turn without requiring user re-specification of search parameters;f. integrate, by said orchestration layer, real-time availability data from said PMS API, static semantic knowledge from vector embeddings of property documentation, and dynamic geographic context from location-based services; andg. synthesize, by a response synthesis module, a comprehensive natural language response incorporating said expanded search results, said semantic knowledge, and said geographic context, all within said single conversational turn.

2.1. The system of claim 1, wherein classifying unavailability comprises:determining partial unavailability when a count of available date objects in the availability response array is greater than zero but less than a total number of requested nights; anddetermining complete unavailability when the count of available date objects in the availability response array is equal to zero.

2.2. The system of claim 1, wherein conditionally triggering the search expansion strategy comprises:triggering a narrow expansion window of \u00b17 days from an original date range for said partial unavailability; andtriggering a broad expansion window of \u00b130 days from the original date range for said complete unavailability.

2.3. The system of claim 1, wherein integrating said static semantic knowledge and said dynamic geographic context comprises:performing a cosine similarity search against said vector embeddings of property documentation to retrieve semantically relevant property information; andapplying rule-based distance calculations to configured geographic coordinates of a property to dynamically integrate location-based contextual services.

3. A computer-implemented method for enhancing vacation rental booking efficiency, comprising:receiving, by a server processor, a natural language booking request from a user;executing, by an API integration module, a real-time query against a Property Management System (PMS) API based on said natural language booking request;analyzing, by a progressive search algorithm module, an availability response array from said PMS API to classify unavailability as one of partial unavailability or complete unavailability;autonomously triggering, by an orchestration layer, a conditional search expansion strategy based on said classification, without requiring user re-specification of search parameters;executing, by said API integration module, a re-query of said PMS API based on said conditional search expansion strategy; andsynthesizing, by a response synthesis module, a natural language response incorporating results from said re-query within a single conversational turn.

3.1. The method of claim 1, wherein analyzing said availability response array

comprises:counting date objects within said availability response array where a status field equals "AVAILABLE";classifying said unavailability as partial unavailability if said count is greater than zero but less than a total number of requested nights; andclassifying said unavailability as complete unavailability if said count equals zero.

3.2. The method of claim 1, wherein autonomously triggering a conditional search expansion strategy further comprises:triggering a narrow expansion window of ±7 days from an original date range for said partial unavailability; andtriggering a broad expansion window of ±30 days from said original date range for said complete unavailability.

3.3. The method of claim 1, wherein said orchestration layer implements workflow branches as conditional execution paths based on an IF-THEN evaluation of said availability classification, and wherein all search expansion, context retrieval, and response synthesis are executed entirely within a single request-response cycle.

4. A computer-implemented method for progressive conversational search and synthesis, comprising:receiving, by a server processor, a natural language booking request;parsing, by a natural language processing module executed by the server processor, said natural language booking request to identify a temporal entity;executing, by an API integration module, a real-time query against a Property Management System (PMS) API based on said temporal entity, thereby obtaining a heterogeneous PMS API response;normalizing, by a data normalization module, said heterogeneous PMS API response into a standardized JSON schema using a property-specific JavaScript transformation function, wherein said standardized JSON schema comprises five mandatory fields including a date field, a status field, a price_amount field, a currency field, and a minimum stay requirement field;analyzing, by a progressive search algorithm module, an availability response array within said standardized JSON schema to perform a real-time availability classification; andautonomously triggering, by an orchestration layer, a conditional search expansion strategy based on said real-time availability classification, all within a single request-response cycle without requiring user re-specification of search parameters.

4.1. The method of claim 1, wherein the status field of said standardized JSON schema comprises enumerated values selected from AVAILABLE, BOOKED, and BLOCKED.

4.2. The method of claim 1, wherein said property-specific JavaScript transformation function is configured to map diverse response formats from different PMS APIs into said standardized JSON schema.

4.3. The method of claim 1, wherein said real-time availability classification comprises: classifying unavailability as partial unavailability when a count of date objects in said availability response array with a status field equal to "AVAILABLE" is greater than zero but less than a total number of requested nights; and classifying unavailability as complete unavailability when

said count of date objects is equal to zero.

5. A computer-implemented method for enhancing vacation rental booking efficiency, the method comprising:receiving, by a server processor, a natural language booking request;executing, by an API integration module, a real-time query against a Property Management System (PMS) API based on said natural language booking request;analyzing, by a progressive search algorithm module, an availability response array from said PMS API to classify unavailability as one of partial unavailability or complete unavailability;autonomously triggering, by an orchestration layer, a conditional search expansion strategy within a single conversational turn based on said classification, without requiring user re-specification of search parameters;integrating, by said server processor, semantic context from property documentation vector embeddings and dynamic geographic context from location-based services;and generating, by a response synthesis module, a single natural language response incorporating expanded search results and said integrated semantic and geographic context.

5.1. The method of claim 1, wherein classifying unavailability as partial unavailability comprises determining a count of available date objects in said availability response array is greater than zero but less than a total number of requested nights, and wherein said conditional search expansion strategy triggers a narrow expansion window of ±7 days from an original date range; and wherein classifying unavailability as complete unavailability comprises determining said count of available date objects is zero, and wherein said conditional search expansion strategy triggers a broad expansion window of ±30 days from said original date range.

5.2. The method of claim 1, further comprising, prior to analyzing said availability response array, normalizing heterogeneous data from said PMS API into a standardized JSON schema comprising a date field, a status field, a price_amount field, a currency field, and a min_stay_requirement field, using property-specific JavaScript transformation functions.

5.3. The method of claim 1, wherein said autonomously triggering is performed by an orchestration layer configured to conditionally execute workflow branches based on an IF-THEN evaluation of said availability classification, and wherein said integrating semantic context and dynamic geographic context occurs concurrently with said conditional search expansion within said single conversational turn.

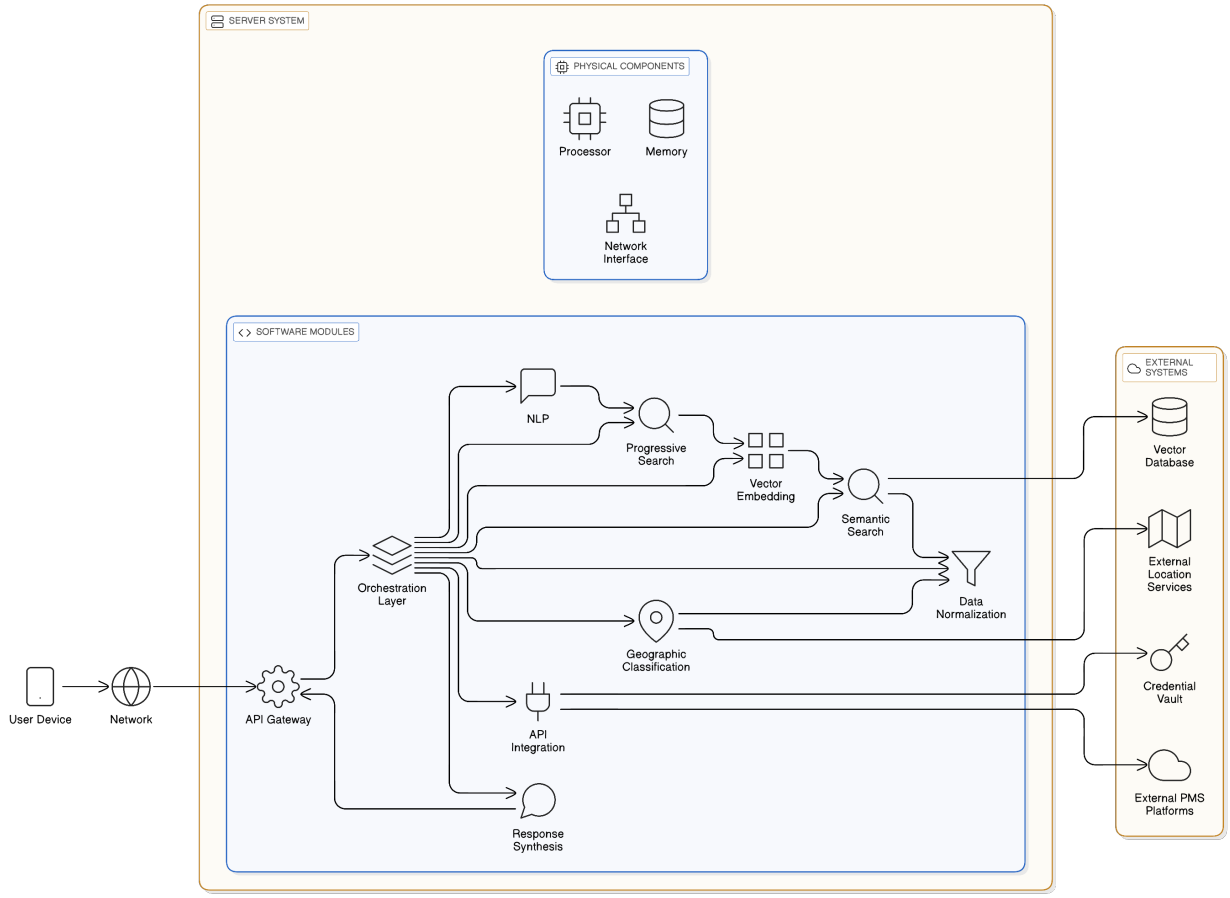
ABSTRACT

A system for conversational booking comprises a processor and memory storing instructions to enhance vacation rental booking efficiency. The system receives a natural language booking request and executes a real-time query against a Property Management System (PMS) API. An availability response array from the PMS API is analyzed to classify unavailability as partial or complete. A conditional search expansion strategy is autonomously triggered based on this classification, re-querying the PMS API with a narrow expansion window for partial

unavailability or a broad expansion window for complete unavailability. The system synthesizes a natural language response incorporating results from the re-querying. The system normalizes API response data into a standardized schema and transforms property documentation into vector embeddings. Cosine similarity searches against these embeddings enable property matching. Geographic coordinates are utilized for location-based contextual services. All operations occur within a single conversational turn.

TECHNICAL DIAGRAMS

1. System Environment for Conversational Booking



2. Key Data Structures for Conversational Booking System

3. Conversational Booking System Operational Workflow